

Data Ingestion Pipeline

Collection & Formatting Stage • Training Material

This document demonstrates the application of the collection and formation stage of the Data Ingestion Pipeline for Retrieval-Augmented Generation (RAG) applications.

Why this matters for RAG:

Extracting structured tables from PDFs is a common challenge. Standard text extraction often scrambles table data, confusing the LLM. By converting the table into a Pandas DataFrame and exporting it as Markdown, you preserve exact row and column relationships in a format that LLMs understand perfectly.

1. Prerequisites

Before running the pipeline, ensure you have the required Python packages installed. You can install them using pip:

```
pip3 install pymupdf pandas tabulate
```

2. Script Usage

The core script, `collection_and_formatting.py`, reads a PDF file containing a table and converts it to a Markdown table using the PyMuPDF package.

3. Python Code Implementation

Save the following code as `collection_and_formatting.py`. The script expects a target file named `Sales_Report.pdf` in the same directory.

```
"""Script to identify and format tables in a PDF file using PyMuPDF."""

import os
import pymupdf # import package PyMuPDF

# get the path of the script
script_path = os.path.dirname(os.path.abspath(__file__))

# construct the path to the PDF file
pdf_path = os.path.join(script_path, "Sales_Report.pdf")

# open the pdf file
pdf = pymupdf.open(pdf_path)

# get the first page
page = pdf[0]

# get the tables in the page
tables = page.find_tables()

# get the first table
table = tables[0]

# convert the table to pandas dataframe
df = table.to_pandas()

# format the dataframe to markdown
markdown_table = df.to_markdown()

# print the markdown table
print(markdown_table)
```

4. Execution

To run the demo and generate the markdown output, execute the following command in your terminal:

```
python3 collection_and_formatting.py
```